Architectural Synthesis of Data Visualization Theory, Bento Grid Systems, and Agentic Frontend Engineering in OpenCode

Executive Synthesis: Bridging Perceptual Theory and Agentic Code Generation

Modern AI-assisted frontend engineering frequently suffers from a structural disconnect between human perceptual science and automated code generation[1]. When a coding agent receives an underspecified request to build an analytical interface, the underlying large language model defaults to statistical averages found in training corpora, producing frosted glassmorphism containers over purple gradients, arbitrary card heights, low-contrast typography, and monolithic component files that conflate server cache with user interface state[2]. Resolving this failure mode requires uniting four distinct engineering layers into a single deterministic workflow that governs both human design intent and autonomous agent execution[3].

At the foundational layer, every visual element must be grounded in Tamara Munzner's *What-Why-How* framework and Four-Level Nested Model for Visualization Design and Validation so that abstract data types and user tasks strictly dictate visual encodings[1]. At the spatial layer, the interface must enforce two-dimensional CSS Bento Grid geometry, mathematically coupled gap-to-radius ratios, single-concept card discipline, and Web Content Accessibility Guidelines (WCAG) 2.1 AA compliance[2]. At the application state layer, frontend code must isolate server cache managed via React Server Components and React Query from shared client-side UI state in Zustand, enforce manual boundary transformations from snake_case to camelCase, and cap component file line counts to prevent architectural drift[3]. Finally, at the agent orchestration layer, these constraints must be encoded into OpenCode's persistent instruction architecture (AGENTS.md), on-demand domain skill modules (SKILL.md), machine-enforced tool permissions (opencode.jsonc), and phased plan-to-build execution workflows[6].

Theoretical Foundations of Information Visualization

Computer-based visualization systems provide visual representations of datasets designed to help humans execute analytical tasks more effectively[1]. Rather than replacing human judgment with black-box computational automation, visualization is required precisely when problems are ill-specified and analysts do not know every question to ask in advance[1]. By shifting the cognitive load to an external visual representation, a well-designed interface replaces slow sequential cognition with high-bandwidth parallel and pre-attentive visual perception, allowing

an operator to experience an immediate simultaneous overview that lower-bandwidth channels like audio cannot support[1]. Furthermore, effective visualization systems represent underlying data distributions rather than relying exclusively on statistical summaries, because summaries strip away granular context needed to confirm expected trends and surface unexpected anomalies[14].

The Four-Level Nested Model and Validation Alignment

In 2009, Tamara Munzner formalized the Four-Level Nested Model for Visualization Design and Validation—subsequently expanded into the Nested Blocks and Guidelines Model—establishing that visualization design operates across four cascading layers where upstream decisions strictly constrain downstream possibilities, and downstream implementation realities trigger upstream iterative refinement[1].

VISUALIZATION ARCHITECTURE • 4-Level Nested Model

Cascading Constraints in Munzner's Four-Level Nested Visualization Model

CORE FOUNDATIONAL RATIONALE

Replace Cognition with High-Bandwidth Parallel Perception

Visual representations augment human capabilities for ill-specified analysis problems where fully automatic algorithms cannot suffice, reaping the pre-attentive bandwidth of the human visual cortex to detect unexpected patterns.

Model Scope: **What • Why • How** (Abstraction Flow)

SELECT ANALYTICAL LENS: Complete Model | What / Why / How | Sales Dashboard | Validation & Threats

↓ Downstream: Cascading Constraints — Click a component title to isolate or inspect — Upstream: Iterative Refinement ↑



1 Domain Situation — Outermost Layer

Core Question: *"Who are the target users and what domain problem do they face?"*

Scope: Domain Problems

THEORETICAL TAXONOMY

Characterize real-world problems in the vocabulary of target domain users. Acknowledge when problem is **ill-specified** so visual discovery augments human intellect rather than blindly automating decisions.

Munzner Premise: Summarize real-world details, science, regression before analysis, pre-attentive pattern hunting.

SALES DASHBOARD CONTEXT

Persona: VP of Global Commercial Operations.
Domain Need: Diagnose why Q3 revenue lags while product sales volume hit targets; determine if reps prioritize low-margin discount contracts.

Business Impact: Clarifies whether compensation incentives distort deal quality across regional fields.

THREAT & VALIDATION — WRONG PROBLEM

Primary Threat: Misunderstanding target user needs (building solutions for the wrong domain challenge).

Valid Method: Conduct field interviews, contextual inquiry, ethnography; observe post-deployment adoption.

Validation Trap: ❌ Post-execution or slick charts can't compensate for absent or useless insights.

↓ Cascading Downward: Domain situation dictates required abstract data types and analytic tasks — Constrains Layer 2

2 Data & Task Abstraction — Level 2: WHAT & WHY

Core Question: *"What data types are operated on, and why are those operations performed?"*

Scope: Generic Taxonomies

WHAT & WHY FRAMEWORK

WHAT: Attributes (quantitative, ordered, categorical) & items.
WHY: Actions (discover, present, locate, identify, compare, summarize) & targets (trends, outliers, correlations, distributions).

Abstraction Role: Converts messy domain jargon into universally comparable visualization operations.

SALES DASHBOARD ABSTRACTION

WHAT Data: Tables with quantitative measures (Revenue, Margin, Volume) keyed by categorical dimensions (Region, Rep, Product Line).
WHY Tasks: *Compare* actuals vs quotas. *Identify* outlier deal discounts. *Correlate* margin with order size.

Operation: Derive profit margin = revenue − quantity to expose unit price degradation.

THREAT & VALIDATION — WRONG ABSTRACTION

Primary Threat: Solving the wrong task; selecting data attributes that hide the target phenomenon.

Valid Method: Test target users on abstract tasks; deploy prototypes to verify task fulfillment.

Validation Trap: ❌ Measuring usability or visual clarity NOT confirming that the abstracted task matches user reality.

↓ Cascading Downward: Data types and task actions restrict viable marks, visual channels, and interactions — Constrains Layer 3

3 Visual Encoding & Interaction Idiom — Level 3: HOW (IDIOMS)

Core Question: *"How are visual marks, graphic channels, and interactive mechanisms arranged?"*

Scope: Perceptual Encoding

HOW IDIOM CHOICES

Visual Marks: Points, lines, areas.
Visual Channels: Spatial position (common scale), length, angle, color saturation/hue.
Interaction: Linked brushing, faceted filtering, semantic zooming, overview + detail.

Perception Law: Channel ranking hierarchy (Cleveland & McGill: Position > Length > Angle > Area).

DASHBOARD IDIOM DESIGN

Idiom: Bullet chart with spatial length against benchmark target bands (instead of unaligned gauges).
Coordination: Coordinated views linking regions' choropleth with multi-dimensional scatterplot (Price vs Volume) and tooltip callouts.

Screen-Space Trade-off: Horizontal grid for regional specifics to conserve vertical height.

THREAT & VALIDATION — INEFFECTIVE IDIOM

Primary Threat: Perceptually misleading encodings, severe visual clutter, unreadable visual idioms.

Valid Method: Controlled lab experiments measuring user error rate and completion time; qualitative perceptual evaluation.

Validation Trap: ❌ Measuring system runtime benchmarks does NOT prove user perception can decode the chart.

↓ Cascading Downward: Visual and interaction idioms establish computational complexity & rendering requirements — Constrains Layer 4

4 Algorithm — Innermost Layer: HOW (CODE)

Core Question: *"How is the visual representation computed and rendered efficiently?"*

Scope: Computational Execution

ALGORITHMIC DESIGN

Design computational pipelines to instantiate chosen idioms; spatial indexing (k-d trees, R-trees), binning aggregations, layout geometry, memory buffering, GPU shaders.

Munzner Theorem: *"Computational benchmarks do not confirm good design or box abstraction."*

FRONTEND ENGINEERING

Data Pipeline: Web Workers processing client-side rollups; IndexedDB caching for 50,000 transaction rows.
Graphics: HTML5 Canvas / WebGL batch instancing to prevent DOM thrash during brush & link interactions.

Optimization: Debounced spatial hit-testing ensuring 60fps buttery scrubbing on mobile screens.

THREAT & VALIDATION — SLOW RUN TIME

Primary Threat: "Your code is too slow" — excessive computational complexity or memory leaks causing stutter.

Valid Method: Profile wall-clock system latency, memory footprint, Big-O asymptotic analysis.

Validation Trap: ❌ Benchmark profiling ONLY validates algorithmic efficiency; it says zero about visual efficacy.

↑ Upstream: Iterative Refinement: Algorithmic limitations force simplifications of visual idioms or data abstractions — Feedback Upward

ARCHITECTURAL SYNTHESIS: WITHIN-LEVEL VS. BETWEEN-LEVEL GUIDELINES

Meyer et al. (IEEE TVCG)

Within-Level Guidelines (Intra-Tier Choices)

Provide guidance for choosing between alternative blocks inside a single level.

Empirical Study: Hao, Kang, & Agrawala (2009) evaluated display grids or bar charts across 4pt to 12pt screen heights, establishing screen-space thresholds for encoding choices within Level 3.

Between-Level Guidelines (Cross-Tier Mappings)

Govern how design blocks match across adjoining levels from abstract types down to channels.

Empirical Study: Heer & Bostock (2010), following Cleveland & McGill (1984), demonstrated perceptual rankings for quantitative data types (Position > Length > Angle > Area), bridging Level 2 to Level 3.

The Epistemic Takeaway for Dashboard Engineers: All potentially within-level evaluations at lower levels rely fundamentally on assumptions made at higher levels. Benchmarking algorithm speed or rendering frame rates tests whether code executes efficiently, but cannot evaluate whether the dashboard addresses real domain needs.

Choices made at higher conceptual tiers strictly constrain downstream engineering blocks, while validation methods must align with the specific tier under evaluation: **computational benchmarks** at the algorithm tier cannot confirm visual idiom effectiveness or task abstraction accuracy.

Data sources: Munzner Nested Model (IEEE TVCG 2009), Meyer et al. (IEEE TVCG), Munzner VAD Tutorial (JHU), Data Books Review, Munzner Lecture Series

At the outermost layer, the **Domain Situation (Level 1)** characterizes the target users, their operational environment, and their workflows using domain-specific vocabulary[1]. Empirical research into applied domains demonstrates how situational constraints shape tooling requirements; for instance, data journalists routinely confront four recurring domain challenges—diachronic, regional, fragmented, and disparate data sources—where visual join interfaces provide substantial utility for practitioners who grasp relational joins conceptually but lack the programming fluency to execute them in code[15]. Because domain design focuses on real-world workflows, validation at this outermost tier relies on anthropological and ethnographic field methods, observing target users after deployment and measuring sustained adoption rates[1].

Nested directly inside the domain context is **Data and Task Abstraction (Level 2)**, which answers the *What* and *Why* questions of the analytical framework by translating domain-specific needs into abstract data structures and generic operations[8]. Data is treated as a structured abstraction of reality recorded in specific formats, while task abstraction defines why the user is examining that data[8]. Specifically, task abstraction establishes whether the operator's objective is to discover new phenomena, present known findings, or enjoy casual exploration, and whether the required analytical actions involve identifying, comparing, annotating, summarizing, aggregating, locating, navigating, selecting, or filtering targets to evaluate trends, outliers, similarities, and correlations[1]. Validation at this second tier confirms that the abstracted tasks and data transformations genuinely solve the domain problem[14].

The third tier, **Visual Encoding and Interaction Idiom (Level 3)**, operationalizes the *How* by mapping abstract data and tasks into visual marks and perceptual channels, governing spatial order, alignment, temporal progression, and interactive multi-view coordination[8]. In complex relational domains such as graph analysis, systems like CorGIE (evaluated with Graph Neural Network experts) or topological trajectory encodings (such as Hop-Census polylines and Census-Census trajectories evaluated across 72 diverse networks) demonstrate that coordinated interactive views reveal structural patterns inaccessible through any single static view alone[15]. Validation at this third level belongs to cognitive and perceptual psychology, verifying through controlled human-subjects experiments that the visual system accurately decodes the chosen visual idiom[1].

At the innermost core sits the **Algorithm (Level 4)**, encompassing software engineering, state synchronization, layout algorithms—such as Glimmer for GPU-based multidimensional scaling, H3 for laying out large graphs in 3D hyperbolic space, or Geomview for 3D geometric topology—and rendering performance[1]. Validation at this innermost level relies on computer science benchmarks that measure computational complexity, wall-clock execution speed, and memory footprint[1].

A foundational principle of the Nested Model is avoiding the **cross-level validation trap**: computational benchmarks at the Algorithm level (for example, demonstrating that a React dashboard renders in 12 milliseconds) *do not confirm idiom design or task abstraction*[1]. An application can achieve optimal algorithmic runtime while failing completely at Level 3 by selecting an illegible visual encoding, or at Level 2 by computing a metric that does not match

the user's analytical task[1]. Consequently, visualization architecture relies on two distinct categories of actionable guidelines[5]. **Between-level guidelines**, which dominate applied design studies, justify how blocks match across tiers—exemplified by Heer and Bostock's empirical rules (building on Cleveland and McGill) for mapping abstract data types in Level 2 to perceptual encoding channels in Level 3[5]. Conversely, **within-level guidelines** arise from technique-driven evaluations comparing blocks inside a single tier, such as Heer, Kong, and Agrawala's empirical screen-space thresholds for choosing between horizon graphs and bar charts across varying display resolutions[5]. Even within-level algorithmic benchmarks remain implicitly coupled to upstream assumptions about the data and task blocks under test[5].

Operationalizing What-Why-How in Dashboard Design

When designing an enterprise analytical workspace—such as a commercial sales, revenue, and operations dashboard—every visual widget must undergo an explicit *What-Why-How* decomposition prior to writing component code[8]:

Analytical Dimension	What (Data Abstraction)	Why (Task Abstraction)	How (Visual Encoding & Interaction Idiom)
Sales Volume	Count of discrete sales orders vs. unit quantities sold per product across geographical regions[8].	Track regional sales team execution or verify whether newly launched products meet baseline adoption targets[8].	Delta-first KPI card with conditional formatting for executive scanning, or a bullet chart plotted directly against a quota target[8].
Revenue Performance	Aggregate monetary value per sales order vs. unit price multiplied by quantity across product lines[8].	Compare revenue against sales volume to identify high-value, low-volume products where sales effort yields superior return, or verify target attainment[8].	Dual-series Recharts bar or column composition or small-multiple comparative views aligned on a shared quantitative scale[8].
Profitability & Margin	Net margin derived from revenue minus cost of goods sold	Identify negative-margin outliers, temporal	Diverging color scale or multi-panel stacked column

	and regional operating overhead[8].	erosion, or regional variance across product cohorts[8].	and progress bar encodings paired with a sortable line-item data table[2].
Network & Relational Flows	Multi-hop graph topologies, Hop-Census polylines, and Census-Census trajectories across disparate sources[15].	Discover structural patterns and topological shifts inaccessible through any single static view alone[15].	Coordinated multiple views with interactive brushing, selection, and filtering across linked sub-panels[1].

Spatial Architecture: Bento Grid Mathematics, Card Discipline, and Accessibility

Once the *What-Why-How* taxonomy establishes which visual idioms belong in the interface, those idioms must be organized into a coherent spatial hierarchy[8]. In modern web engineering, the **Bento Grid** serves as the premier layout system for dashboards, feature showcases, and multi-metric workspaces because it divides the viewport into modular rectangular tiles of varying sizes where heterogeneous content blocks coexist with clear hierarchy, soft spacing, and subtle contrast without competing for visual attention[4].

The Five Structural Laws and Grid Mathematics of Bento Layouts

A true Bento Grid is governed by strict mathematical rules closer to a spreadsheet than freehand visual design[7]. At the CSS implementation layer, a bento dashboard must always be built on two-dimensional **CSS Grid** (display: grid), never one-dimensional Flexbox, because Flexbox arranges items along a single row or column whereas bento tiles must span multiple columns and rows simultaneously[4]. Beyond two-dimensional grid tracks, an authentic bento architecture enforces five structural laws[7]. First, the layout must exhibit uneven cell sizes on a single grid, incorporating at least two distinct cell footprints (such as 1×1 and 2×1) so the layout does not degenerate into a uniform card grid[7]. Second, every cell must share a single unified gap value and corner radius system so human vision perceives the tiles as one cohesive object rather than unrelated floating cards[7]. Third, every cell height must align to a shared base row height multiple, keeping the bottom horizon of the grid flat despite varying internal tile heights[7]. Fourth, spatial size must encode analytical importance, meaning the largest hero tile holds the primary proof point rather than whichever card happens to contain the longest text copy[7]. Fifth, the layout must comprise a closed, fixed set of cells with a deterministic start and end; an infinitely scrolling or dynamically appended container is a feed rather than a bento grid[7].

Mathematically, the grid starts with an 8px or 4px base spacing unit (U), and every cell width and height is a whole multiple of that unit[7]. On a standard 12-column desktop CSS Grid (grid-template-columns: repeat(12, 1fr)), a 4-unit bento assigns 3 page columns per bento unit with a base row height of 96px to 104px, whereas a 6-unit bento assigns 2 page columns per bento unit[4]. Engineers must rigorously separate **cell footprints**—the number of bento units a cell spans, such as 1×1, 2×1, 1×2, 2×2, and occasionally 3×1—from **aspect ratios**, which describe the geometric proportion rendered on screen[7]. A 1×1 footprint renders at a $1 : 1$ square aspect ratio, a 2×1 footprint renders at $2 : 1$, and a 1×2 footprint renders at $1 : 2$[7]. Crucially, a 2×2 hero footprint also renders at a $1 : 1$ square aspect ratio rather than introducing a fourth ratio family, keeping the entire layout constrained to three core ratio families ($1 : 1$, $2 : 1$, and $1 : 2$, with $3 : 1$ reserved for occasional wide banners)[7].

Furthermore, the inter-tile gap (G) and card corner radius (R) are mathematically linked rather than independent stylistic choices, governed by the interval[7]:

$$0.5R \leq G \leq 1.0R$$

In concrete terms, a 24px corner radius pairs with a 12px to 20px grid gap, while a 16px corner radius pairs with an 8px to 14px gap[7]. Across responsive viewports, CSS media queries reflow the grid systematically: a single-column stack (grid-template-columns: 1fr) on mobile screens, a 2-column matrix (repeat(2, 1fr) with grid-auto-rows: minmax(180px, auto)) at 600px tablet widths, and a 4-column (repeat(4, 1fr)) or 12-column (repeat(12, 1fr)) grid with explicit row tracks or grid-template-areas at 900px+ desktop viewports[4].

FRONTEND ARCHITECTURE STANDARD 12-Col • WCAG 2.1 AA • Clean Code

Quantitative Engineering Specifications for Bento Grids, Accessibility, and Component Sizing

Panel 1: Mathematical Geometry & Footprint Ratios (12-Col / 4-Unit Grid)

Row Height: 96–104px | Base: 8px / 4px

Click any cell in the 12-column wireframe to inspect its exact spatial footprint, rendered aspect ratio, CSS Grid spans, and equivalent Tailwind utilities.

grid-template-columns: repeat(12, 1fr) — 1 Bento Unit = 3 Columns

- **2×2 Hero Tile** — 1:1 Ratio — col-span-6 • row-span-2
- **2×1 Wide** — 2:1 Ratio — col-span-6 • row-span-1
- **1×1 Std** — 1:1 — col-span-3
- **1×2 Tall** — 1:2 — col-span-3 • row-span-2
- **3×1 Banner** — 3:1 Ratio — col-span-9 • row-span-1

SELECTED FOOTPRINT: 2×2 Hero (Primary Proof Point) — Rendered Ratio: 1:1 Square

GRID GEOMETRY	TAILWIND CLASS	CSS DECLARATION
6 cols × 2 rows (2 units)	col-span-6 row-span-2	grid-column: span 6;

Radius 24px System — Gap: 12px – 20px

Formula: `radius / 2 ≤ gap ≤ radius`. 24px corner radius guarantees internal visual alignment when paired with 16px desktop padding.

Radius 16px System — Gap: 8px – 14px

Formula: `radius / 2 ≤ gap ≤ radius`. 16px corner radius pairs strictly with 8px, 12px, or 14px grid gutters to prevent perceptual card detachment.

Panel 2: Color Tokens & WCAG 2.1 AA Accessibility Guardrails

Contrast: 4.5:1 (Normal) | 3.0:1 (Large)

Primary	Secondary	Success	Warning	Dark Surface	Focus Ring
#FAD4C0	#88A1C1	#16A34A	#D97706	#09090b	#0066CC
Warm Accent	Supporting	Valid States	Attention State	Base Canvas	A11y Outline

Normal Text Contrast — ≥ 4.5:1

Mandatory for all body text <18pt (24px). Dark mode bento tiles require rigorous testing against #09090b to prevent contrast failures.

Large Text / UI Icons — ≥ 3.0:1

Permitted exclusively for headers ≥18pt or ≥14pt bold. Applied to prominent metric numbers and hero KPI values within bento cards.

Keyboard Focus Ring — 2px Offset

Tab or Click to Test Focus

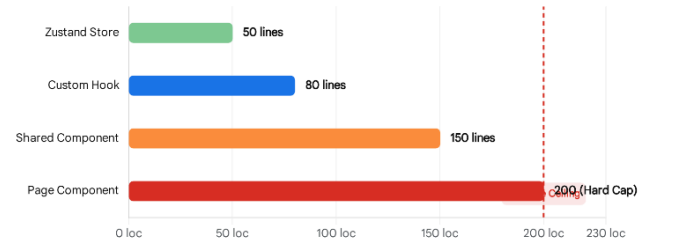
```
:focus-visible { outline: 2px solid #0066CC; outline-offset: 2px; }
```

Panel 3: Modular React / TypeScript File Sizing Ceilings

Strict Ceiling: 200 Lines Max

Maximum permissible lines of code before mandatory component extraction:

Store (≤50) Hook (≤80) Shared (≤150) Page (≤200)



Zustand Store (≤50)
Stores only UI state. If over 50 lines, server state is leaking into client store.

Custom Hook (≤80)
Isolate single-purpose domain behavior; split query logic when exceeding 80 lines.

Shared Component (≤150)
Reusable UI blocks; extract sub-components into the local folder if exceeded.

Page Component (≤200)
Hard ceiling. Never generate any single file exceeding 200 lines.

Strict mathematical coupling between corner radius and grid gap preserves visual cohesion at the layout layer, while **WCAG 2.1 AA** contrast thresholds and **200-line** maximum file constraints prevent accessibility debt and monolithic component bloat.

Data sources: Frontend Architecture Prompt, CodeShack Bento Layout, TypeUI Bento Design Skill, Orbix Studio Bento Guide, SetProduct Bento System

Card Discipline, KPI Anatomy, and Color Systems

To prevent AI-generated cards from looking generic, design specifications must explicitly override model defaults by prohibiting frosted glassmorphism boxes over purple gradients, decorative gradient-blob icons, filler copy, and multiple equal-weight ghost buttons[2]. Following Nielsen Norman Group usability standards, a card is a short, linked representation of a **single conceptual unit** that performs one job and exposes at most **one primary action**[2]. Because the card acts as an entry point to deeper inspection, the entire card surface usually serves as the interactive click target, pushing secondary actions to the detail view[2]. Content inside each tile must follow a strict visual reading order: primary visual or metric first, a concise title second, a muted line of metadata third, and the single primary action last, with only the primary action receiving accent color[2]. When operators need to scan uniform rows or compare items line by line across shared attributes, cards are less scannable than lists and must be replaced by a structured Data Table[2].

For quantitative monitoring inside compact 1×1 cells, **delta-first KPI cards** pair the primary metric and period-over-period delta badge with an unopinionated slot for a lightweight **SVG sparkline**, keeping the metric immediately interpretable without loading heavy client-side charting engines[17]. Larger 2×1 or 2×2 analytical tiles utilize **shadcn/ui** chart components built on **Recharts** (optionally enhanced with Framer Motion animations via libraries like Easy UI), which inherit theme colors automatically and provide responsive axes, multi-series tooltips, and legends[18]. Internal card sections, toolbars, and sidebars are divided using the Shadcn Separator primitive—supported across both Radix UI for structured accessibility and Base UI for lower-level customization—to organize complex layouts without visual clutter[21].

Typographic hierarchy within a bento system starts at 12px to accommodate compact 1×1 cells where space is constrained and caps headings at 32px so even the largest titles remain proportional to the grid, utilizing font weights from 100 to 900 in a contemporary neo-grotesque typeface[10]. Rather than relying on sterile gray defaults, bento color systems combine intentional surface tokens with colorblind-safe data scales[10]:

Palette System / Token	Hex / Scale Specification	Architectural Purpose & Perceptual Role
Warm Bento — Primary	#FAD4C0	Primary interactive elements, highlighted bento blocks, and warm accent surfaces that avoid cold gray sterility[10].

Warm Bento — Secondary	#80A1C1	Supporting actions, secondary highlights, and complementary visual accents[10].
Warm Bento — Semantic States	#16A34A (Success) / #D97706 (Warning)	Positive feedback, valid states, and upward delta badges (#16A34A) vs. caution indicators and attention prompts (#D97706)[10].
Dark Graphite Surface	#09090b (Canvas) / #FFFFFF (Text)	High-contrast dark-mode dashboard canvas paired with crisp white typography and single-accent discipline[2].
Okabe-Ito Accessible Scale	Plain vector of colorblind-safe hex codes	Categorical data visualization palette ensuring multi-series charts remain distinguishable across color-vision deficiencies[27].

WCAG 2.1 AA Accessibility Requirements for Bento Dashboards

Every production bento grid must satisfy six mandatory WCAG 2.1 AA engineering requirements so spatial modularity does not degrade assistive navigation[4]. First, keyboard tab order must follow the visual reading order by structuring the HTML DOM in the exact sequence rendered by the CSS Grid; using tabindex values above 0 to force tab order is strictly prohibited because it creates compounding accessibility debt whenever the grid layout changes[4]. Second, every tile requires an explicit aria-label on the outer container element itself—such as aria-label="Monthly Recurring Revenue: $142,000"—rather than solely on the inner numeral, preventing screen readers from announcing bare numbers stripped of semantic meaning[4]. Third, color contrast must achieve at least 4.5:1 for normal text and 3:1 for large text, verified using Stark, Chrome DevTools, or axe-core; dark-mode glassmorphism implementations frequently fail this threshold by placing light gray text over translucent dark gray tiles[4].

Fourth, interactive tiles that link to detail views or trigger actions must restore a visible keyboard focus ring via .tile:focus-visible { outline: 2px solid #0066CC; outline-offset: 2px; }, using :focus-visible rather than :focus so the ring guides keyboard users without appearing on mouse clicks[4]. Fifth, dashboards containing 10 or more tiles must group related widgets into

landmark sections using role="region" with descriptive aria-label attributes and provide top-level skip-navigation links so keyboard users do not have to tab through 16+ individual tiles to reach target content[4]. Sixth, tiles displaying live-updating telemetry—such as ticking revenue counters or active alert totals—must apply aria-live="polite" so screen readers announce updates during natural pauses, reserving aria-live="assertive" exclusively for critical system emergencies that justify interrupting the user mid-sentence[4].

Frontend Software Architecture and State Management Guardrails

To ensure that an AI coding agent generates maintainable production code rather than monolithic files, the system prompt must enforce a deterministic frontend architecture[3]. The foundational invariant of this architecture is that **Server State is never mixed with Client UI State**[3].

State Management Decision Protocol and Anti-Patterns

Every datum entering the frontend application must be routed through a deterministic four-branch state decision tree[3]. When data originates from an API and represents initial page load content requiring no client-side interactivity, it must be fetched inside **React Server Components (RSC)** using native async/await so zero client JavaScript is shipped to the browser[3]. When API data requires client-side interactivity, caching, background refetching, or pagination, it must be managed exclusively through **React Query** (or SWR) using a structured per-domain **Query Key Factory** (all, lists(), list(params), details(), detail(id)), an explicit staleTime (such as 30_000 milliseconds), and automatic cache invalidation (queryClient.invalidateQueries()) inside onSuccess mutation callbacks[3]. Conversely, when state does not originate from an API and is shared across unrelated UI components (such as selectedTab or isSlideOverOpen), it belongs in a lightweight **Zustand** (or Pinia) store[3]. Finally, state isolated to a single component uses local useState, while form state is kept strictly local inside a form library (React Hook Form or Formik) paired with schema validation (Zod or Yup)[3]. A critical architectural anti-pattern is placing API data (items[], isLoading, error, or fetchX() functions) or form state inside a Zustand store[3]. Re-implementing server cache management inside Zustand strips away request deduplication, background refetching, and cache invalidation while introducing race conditions[3].

Domain-Colocated Directory Architecture and Boundary Transformation

All business logic must be colocated by domain vertical inside src/lib/<domain>/, cleanly separated from Next.js App Router pages (src/app/), shell layouts (src/components/layout/), reusable feature components (src/components/shared/), and UI primitives (src/components/ui/)[3]. Crucially, all payload transformations between backend snake_case schemas and frontend camelCase models must occur manually at the API boundary inside transformers.ts[3]:

Architectural Path	Responsibility & Engineering Rules	Hard Sizing / Structural Constraint
src/app/(dashboard)/<role>/	Role-scoped dashboard routes (config.ts, layout.tsx, page.tsx), BFF route handlers (src/app/api/), and single-source design tokens (globals.css)[3].	**Page Component**: Maximum ~200 lines; extract sub-components or hooks if exceeded[3].
src/components/shared/<name>/	Reusable feature components (e.g., Bento KPI cards, Recharts wrappers, Data Tables), one component per kebab-case folder with index.tsx[3].	**Shared Component**: Maximum ~150 lines; split barrel exports (index.ts) when exports exceed 20[3].
src/lib/<domain>/types.ts	Defines raw backend payload interfaces (Backend* in snake_case) alongside clean frontend domain models (in camelCase)[3].	Never leak Backend* types into UI components[3].
src/lib/<domain>/transformers.ts	Pure manual converter functions mapping snake_case backend payloads to camelCase frontend entities at the API boundary[3].	Manual mapping only (no reflection auto-mappers); the *only* file where backend field names may appear[3].
src/lib/<domain>/api.ts	Service functions invoking the stateless singleton apiClient, piping raw responses through transformers.ts, and returning typed frontend models[3].	Stateless HTTP execution; no tokens stored in client state[3].
src/lib/<domain>/hooks.ts	Domain Query Key Factory (<domain>Keys) plus useQuery and useMutation	**Custom Hook**: Maximum ~80 lines; split into smaller hooks if exceeded[3].

	wrappers[3].	
src/lib/<domain>/store.ts	Optional Zustand store holding shared UI-only state (selectedMetricId, isFilterDrawerOpen)[3].	**Zustand Store**: Maximum ~50 lines; exceeding 50 lines indicates server state contamination[3].

The OpenCode Agentic Ecosystem and Context Engineering

To execute these visualization and architectural standards autonomously, the prompt engineering strategy must align with the internal mechanics of **OpenCode** (anomalyco/opencode), an MIT-licensed open-source AI coding agent with 225,000 GitHub stars and 30,200 forks[11].

Evolution and Architecture of OpenCode V2

While OpenCode V1 (opencode-ai/opencode) was originally built as a Go-based CLI application using Cobra, a Vim-like editor, LSP integration, and SQLite session storage[30], **OpenCode V2** (anomalyco/opencode, release v2.2.5) is a comprehensive multi-platform agentic platform written in TypeScript (81.6%), MDX (14.6%), CSS (2.4%), Rust (1.0%), Astro (0.2%), and JavaScript (0.1%), powered by Bun, Effect's official Bun and Node SQLite clients, and @opencode/pty[11]. OpenCode V2 runs across modern terminal emulators (WezTerm, Alacritty, Ghostty, and Kitty), a native Desktop app (macOS Apple Silicon/Intel, Windows, and Linux .deb, .rpm, and .AppImage), a paired Web interface (opencode pair), a VS Code extension supporting V2 line-range mention syntax (@path#37-42), Docker images (ghcr.io/anomalyco/opencode:2.0.0), and GitHub Actions workflows triggered via /opencode or /oc comments in pull requests and issues[11]. Providers can be connected via /connect (supporting Anthropic, OpenAI, Google Gemini, Groq, Ollama, OpenCode Console, or the $10/month OpenCode Go subscription) or extended through Model Context Protocol (MCP) servers over Stdio and Server-Sent Events (SSE)[30].

Internally, OpenCode enforces a strict dependency hierarchy (Schema to Core and Protocol, then to Server, with sdk composing Client, Core, and Server) and separates durable prompt admission (session.inbox.enqueued persisted in session_inbox) from process-local model execution (SessionRunner, where each step is one explicit llm.stream call continuing from reloaded history without an in-memory tool loop)[35]. During extended coding tasks, OpenCode's Auto-Compact system monitors token consumption and automatically summarizes the session when usage reaches **95% of the model's context window**, initializing a new continuation session to prevent out-of-context failures[30].

OPENCODE V2 CORE | Deterministic Assembly | 95% Auto-Compact

OpenCode V2 Context Assembly Pipeline and Repository Architecture

Strict concatenation order, dual configuration surfaces, and codebase composition of `anomalyco/opencode`.

V2 Migration Rule:
OpenCode V2 recognizes **`AGENTS.md`** only. The ~~`CLAUDE.md`~~ fallback has been removed.

SURFACE COMPARISON

Complementary, Non-overlapping

AGENTS.md — BEHAVIORAL CONTEXT

Direct Markdown instructions concatenated into system prompt. Governs coding conventions, guidelines, and judgment.

Precedence	All files load; none override
Discovery Scope	`Global → Workspace → Root`
Live Updates	Automatic before next prompt

opencode.jsonc — MACHINE GATING

Structured machine configuration. Governs tool access, execution permissions, provider settings, and plugins.

Precedence	Local overrides; permissions append
Format	`JSONC (comments & trailing commas)`
Reload Trigger	Read on startup (Restart needed)

6-Stage Instruction Assembly Pipeline

Sequential injection into model context (Combined, never overridden). Click stage for deep-dive.

`AGENTS.md edit → Update → Next Prompt`

1. **Agent / Provider System Prompt** — Core Base
   Active agent definition (build vs plan) and underlying provider system...
2. **Code Mode Tool Guidance** — Conditional
   Active when Code Mode is enabled; equips agent with programmatic ...
3. **MCP Server, Reference & Skill Guidance** — Lazy Loaded
   External MCP protocols and on-demand `SKILL.md` definitions loaded...
4. **Global & Project AGENTS.md + Config** — Multi-file
   Concatenates `~/.config` down to workspace, then globs/URLs.
5. **Built-in Date & Environment Context** — Runtime
   System timestamps, active OS architecture, shell variables, and execu...
6. **Session-Specific API Instructions** — Terminal/API
   Custom session prompt overrides and terminal client flags supplied at...

1 Agent / Provider System Prompt

Order: 1 of 6

FILE & DISCOVERY PATH

```
Internal Core: Agent definitions
(build / plan / subagents)
```

PRECEDENCE & COMBINING SEMANTICS

Sent first to establish foundation. Base permissions set default read-write (build) or read-only (plan).

LIVE-RELOAD & CONTEXT RULE

Switched via Tab key in TUI. Provider settings persist across sessions.

Context Gauge: 95% Auto-Compact

95% Limit

0% (Empty) — 95% Summarize & Roll — 100%

OpenCode monitors token consumption continuously. Reaching **95%** triggers automatic session summarization, creating a fresh session without context loss.

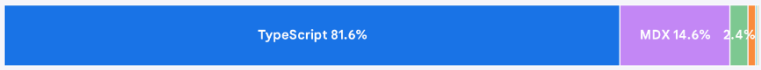
Codebase Language Composition

Source distribution of `anomalyco/opencode`

★ 225k Stars | 30.2k Forks | v2.5

LANGUAGES

TypeScript: 81.6% | MDX: 14.6% | CSS: 2.4% | Rust: 1.0% | Astro: 0.2% | JS: 0.1% | Other: 0.1%



0% Baseline — `Dominant Engine: Bun/Node & Effect-TS` — 100% Total Codebase

OpenCode V2 concatenates six instruction layers into the model context without overriding prior layers, while offloading domain-specific workflows to on-demand **SKILL.md** files preserves token budget below the **95%** auto-compaction threshold.

Data sources: anomalyco/opencode (GitHub), OpenCode Documentation, Dev Genius, Grigio.org, OpenCode CLI Repository

Dual Configuration Surfaces and Lazy-Loaded Skills

OpenCode is configured through two complementary surfaces that cannot replace one another: **AGENTS.md** shapes how the agent judges by feeding Markdown instructions straight into the model's context, whereas **opencode.json / opencode.jsonc** gates what the agent is permitted to do via machine-parsed settings for permissions, agents, providers, and plugins[6]. Stewarded by the Agentic AI Foundation under the Linux Foundation, AGENTS.md acts as a dedicated, predictable README for AI agents across repositories[36]. While OpenCode V1 supported CLAUDE.md as a fallback rule file, **OpenCode V2 recognizes AGENTS.md exclusively**[13]. When a session initializes, OpenCode assembles initial instructions in a strict six-stage concatenation sequence: (1) the selected agent or provider system prompt, (2) Code Mode tool guidance when enabled, (3) MCP server, reference, and skill guidance, (4) global (~/.config/opencode/AGENTS.md) and project AGENTS.md files traversed from the workspace directory up to the project root followed by instructions entries from opencode.jsonc, (5) built-in date and environment context, and (6) session-specific API instructions[6]. Every discovered AGENTS.md file loads and concatenates without overriding earlier files, and any mid-session edit to AGENTS.md is automatically injected as an instruction update prior to the next prompt[6]. Conversely, opencode.jsonc files are evaluated from low to high priority where later keys win on collisions and permission rulesets append[6].

A crucial second-order consequence of this architecture is the risk of **context window pollution**. If exhaustive rules for CSS Bento grids, Recharts props, and WCAG attributes are hardcoded directly into the root AGENTS.md, every unrelated backend or database prompt pays that token overhead, accelerating the session toward the 95% auto-compaction boundary[30]. To preserve context budget, engineers should keep the root AGENTS.md focused on repository-wide state and file-sizing invariants while placing specialized visualization rules inside a project skill file (.opencode/skill/bento-viz-dashboard/SKILL.md with YAML frontmatter) that OpenCode lazy-loads on demand via its Read tool only when executing frontend design tasks[12].

Multi-Agent Execution: Plan Mode, Multimodal Input, and Hybrid Review

OpenCode includes two primary built-in agents that developers toggle between using the Tab key, alongside an internal @general subagent for complex multi-step searches[11]. The **plan agent** operates in a read-only analysis mode that denies file edits by default and requests explicit permission before running bash commands, making it ideal for exploring codebases and validating architecture before touching code[11]. The **build agent** serves as the default full-access agent for executing file modifications and running verification suites[11].

When engineering a complex visualization dashboard, operators should always begin in **plan mode** to validate Levels 1 through 3 of Munzner's Nested Model (domain metrics, data/task abstractions, and Bento footprint allocations) and can drag and drop reference design images ([Image #1]) directly into the terminal for multimodal scanning before hitting Tab to switch to **build mode**[1]. Once build mode finishes generating the components, teams can run **Alibaba's Open Code Review (ocr)** (alibaba/open-code-review), a hybrid static-pipeline and LLM code

review CLI tool benchmarked against 1,505 annotated ground-truth defects in AACR-Bench that integrates natively with OpenCode slash commands and Delegation Mode to verify precision, recall, security (XSS, SQL injection, null-pointer safety), and architectural compliance[38].

Production Prompt Suite for OpenCode

The following four production artifacts provide a complete, drop-in configuration and prompt suite for OpenCode V2. Together, they operationalize Munzner's *What-Why-How* taxonomy, the mathematical Bento Grid layout system, WCAG 2.1 AA compliance, and the React Query / Zustand state separation architecture[3].

1. Persistent Repository Instructions (AGENTS.md)

Save the following file as AGENTS.md in the root of the repository and commit it to Git so OpenCode loads these architectural guardrails automatically at Stage 4 of context assembly[13]:

Project Architecture & Agent Instructions (AGENTS.md)

1. Core Architectural Guardrails

- **State Separation Mandate (Server State != Client State)**:
  - Initial static API data with no client interactivity MUST use React Server Components (RSC) with async/await.
  - Dynamic client-side API data MUST use React Query (@tanstack/react-query) with a domain Query Key Factory (all, lists, list, details, detail) and an explicit staleTime (e.g., 30_000).
  - Shared UI-only state across unrelated components MUST use Zustand (store.ts).
  - Local component state MUST use useState. Form state MUST remain local in React Hook Form + Zod.
  - **CRITICAL ANTI-PATTERN**: NEVER place API data (items[], isLoading, error, fetchX()) or form state inside a Zustand store.
- **Domain Colocation (src/lib/<domain>/)**:
  - Group all business logic by domain: types.ts (Backend* in snake_case, Frontend models in camelCase), transformers.ts (manual converters mapping snake_case and camelCase; backend field names must NEVER appear outside types.ts and transformers.ts), api.ts (stateless apiClient calls and transformation), hooks.ts (React Query hooks), store.ts (optional Zustand UI store), and index.ts (tight barrel export, split if exceeding 20 exports).

2. Hard File-Sizing Constraints

- **Page Component (src/app/.../page.tsx)**: Maximum 200 lines. Extract sub-components or hooks if exceeded.
- **Shared Component (src/components/shared/<kebab-case>/index.tsx)**: Maximum 150 lines.

- **Custom Hook (hooks.ts)**: Maximum 80 lines.
- **Zustand Store (store.ts)**: Maximum 50 lines.
- **Absolute Ceiling**: NEVER generate any single file exceeding 200 lines of code.

3. Lazy-Loaded Visualization & Bento Grid Skill

- CRITICAL: When tasked with designing, refactoring, or building UI dashboards, cards, charts, or data visualizations, use the Read tool to load .opencode/skill/bento-viz-dashboard/SKILL.md on a need-to-know basis before generating a plan or writing code.

4. Verification Workflow

- Run bun typecheck and bun lint after modifying any TypeScript or TSX file.
- Do not edit generated client or schema files directly.

2. On-Demand Visualization & Bento Design Skill (.opencode/skill/bento-viz-dashboard/SKILL.md)

Save the following file at .opencode/skill/bento-viz-dashboard/SKILL.md so OpenCode loads detailed visual design rules on demand when working on frontend tasks[10]:

name: bento-viz-dashboard description: Applies Tamara Munzner's What-Why-How visualization taxonomy, mathematical CSS Bento Grid rules, KPI card anatomy, and WCAG 2.1 AA accessibility to React, shadcn/ui, and Recharts dashboards. license: MIT compatibility: opencode metadata: domain: frontend-visualization stack: nextjs-tailwind-shadcn-recharts

1. Tamara Munzner's Nested Model & What-Why-How Decomposition

Before writing UI code, map every dashboard widget through three mandatory tiers:

1. **WHAT (Data Abstraction)**: Specify exact data types and mathematical definitions (e.g., discrete order count vs. unit price multiplied by quantity; raw point-in-time value vs. period-over-period rate of change).
2. **WHY (Task Abstraction)**: Define the explicit analytical action—Identify, Compare, Summarize, Locate, Filter, or Detect Outliers and Trends.
3. **HOW (Visual Encoding & Interaction Idiom)**:
   - Single KPI and micro-trend: Delta-first KPI Card (1x1 cell) with inline lightweight SVG sparkline and conditional delta badge.

- Target attainment: Bullet chart or progress bar against quota.
- Multi-series temporal or categorical comparison: shadcn/ui and Recharts stacked column, bar, or line chart (2x1 or 2x2 hero cell) operating on a single shared Y-axis scale (NEVER use dual-axis combo charts or radar/spider charts).
- Line-by-line record comparison: Sortable Data Table (NEVER use cards for uniform multi-row scanning).

2. Mathematical Bento Grid Specification

- **CSS Grid Foundation**: Always use display: grid (grid-template-columns: repeat(12, 1fr) on desktop >=900px, repeat(2, 1fr) with grid-auto-rows: minmax(180px, auto) at >=600px, and 1fr on mobile). Never use Flexbox for two-dimensional tile layouts.
- **Base Unit & Row Height**: Use an 8px base spacing scale and a shared base row height multiple (96px to 104px per unit row) so the bottom edge of the grid aligns flat.
- **Legal Footprints & Aspect Ratios (4-Unit Bento on 12-Col Grid)**:
  - 1x1 cell (col-span-3 row-span-1, aspect ratio 1:1): Standard KPI metric card.
  - 2x1 cell (col-span-6 row-span-1, aspect ratio 2:1): Wide time-series or comparative chart.
  - 1x2 cell (col-span-3 row-span-2, aspect ratio 1:2): Vertical activity feed or ranked breakdown.
  - 2x2 cell (col-span-6 row-span-2, aspect ratio 1:1): Hero analytical view holding the primary proof point.
- **Gap-to-Radius Coupling Formula**: Set grid gap between 0.5x and 1.0x of the card border-radius:
  - 16px border-radius (rounded-2xl) pairs with 12px or 16px grid gap (gap-3 or gap-4).
  - 24px border-radius (rounded-3xl) pairs with 16px or 20px grid gap (gap-4 or gap-5).

3. Card Restraint & Aesthetic Overrides

- **Override Generic AI Defaults**: Strictly NO frosted glassmorphism over purple gradients, NO decorative gradient-blob icons, and NO multiple equal-weight action buttons inside a single card.
- **Single Concept Rule**: One conceptual unit and at most one primary action per card. Order content strictly: (1) Primary Metric or Visual, (2) Concise Title, (3) Muted Meta or Comparison Context, (4) Single Primary Action or whole-card click target. Use Shadcn Separator for clean internal dividers.
- **Color Palettes & Typography**:
  - Typography: 12px minimum for compact bento labels up to 32px maximum for top headings.
  - Warm Bento Palette: Primary #FAD4C0, Secondary #80A1C1, Success #16A34A, Warning #D97706.
  - Dark Graphite Palette: Canvas #09090b, Surface #121215, Text #FFFFFF.
  - Multi-Series Charts: Bind Recharts series to colorblind-safe Okabe-Ito tokens (#E69F00, #56B4E9, #009E73, #F0E442, #0072B2, #D55E00, #CC79A7).

4. Mandatory WCAG 2.1 AA Accessibility Checklist

- DOM order MUST match visual CSS Grid reading order; NEVER use tabindex > 0.

- Apply full semantic aria-label on the outer tile container (e.g., aria-label="Monthly Recurring Revenue: $142,000, up 12% month over month").
- Enforce >= 4.5:1 contrast for normal text and >= 3:1 for large text.
- Restore keyboard focus rings on interactive tiles: focus-visible:outline-2 focus-visible:outline-offset-2 focus-visible:outline-[#0066CC].
- Group grids with 10+ tiles into <section role="region" aria-label="..."> landmarks with top skip-navigation links.
- Apply aria-live="polite" to streaming data tiles and reserve aria-live="assertive" strictly for critical alerts.

3. Machine Configuration (opencode.jsonc)

Save the following file as opencode.jsonc in the project root to register the instruction files and enforce read-only safety during planning[6]:



4. Interactive Master Prompt for OpenCode (Plan Mode to Build Mode)

Copy and paste the following prompt directly into the OpenCode TUI—beginning in **plan mode** via the Tab key—to analyze a domain dataset and generate a complete, accessible Bento Grid dashboard[11]:

Act as a Principal Information Visualization Architect and Lead React/TypeScript Engineer. Load .opencode/skill/bento-viz-dashboard/SKILL.md and AGENTS.md first. Execute this task in two strict phases matching OpenCode's plan and build workflow:

PHASE 1: ARCHITECTURE & VISUALIZATION PLAN (Execute now in plan mode — DO NOT write files yet)

1. **Munzner Nested Model & What-Why-How Specification**:
   - Inspect the existing domain schemas and API endpoints in the workspace (using @general if multi-file discovery is needed).
   - Output a markdown table mapping every proposed dashboard metric across:
     - **Domain Situation (Level 1)**: Target operator role and decision context.
     - **WHAT (Level 2 Data Abstraction)**: Exact backend field (snake_case), transformed frontend property (camelCase), unit, and aggregation formula.
     - **WHY (Level 2 Task Abstraction)**: Primary analytical verb (Identify, Compare, Summarize, Filter, or Detect Outliers).
     - **HOW (Level 3 Visual Idiom & Bento Footprint)**: Chosen component archetype (Delta-first KPI Card with SVG Sparkline, Recharts Single-Axis Stacked Column/Line Chart, Bullet Progress Bar, or Sortable Data Table), exact 12-column Bento footprint (1x1 / col-span-3 row-span-1, 2x1 / col-span-6 row-span-1, 1x2 / col-span-3 row-span-2, or 2x2 Hero / col-span-6 row-span-2), and aspect ratio (1:1, 2:1, or 1:2).
2. **State & File Decomposition Blueprint**:
   - List every file to be created or updated across src/lib/<domain>/ (types.ts, transformers.ts, api.ts, hooks.ts, store.ts, index.ts), src/components/shared/<kebab-case>/index.tsx, and src/app/(dashboard)/.../page.tsx.
   - Confirm the estimated line count for each file is strictly below the hard limits (Page <= 200 lines, Shared Component <= 150 lines, Hook <= 80 lines, Zustand Store <= 50 lines) and verify zero server state is placed in Zustand.
3. **Accessibility & Aesthetic Verification**:
   - Specify the exact border-radius and coupled grid gap (0.5R <= G <= 1.0R), color tokens (Warm Bento or Dark Graphite plus Okabe-Ito chart series), container-level aria-label templates, :focus-visible (#0066CC) ring classes, role="region" landmarks, and aria-live="polite" attributes.

Pause after outputting Phase 1 so the plan and any attached reference wireframes ([Image #1]) can be reviewed.

PHASE 2: DETERMINISTIC IMPLEMENTATION (Execute after switching

to build mode via Tab)

Once approved to build:

1. Scaffold the domain layer in src/lib/<domain>/ in sequence: types.ts, manual transformers.ts, api.ts, React Query hooks.ts with Query Key Factory and staleTime: 30_000, UI-only Zustand store.ts, and barrel index.ts.
2. Implement the modular Bento Grid container, delta-first SVG sparkline KPI cards, single-axis shadcn/ui and Recharts visualizations, and Shadcn Separator dividers in src/components/shared/ (one component per kebab-case folder, <= 150 lines each).
3. Assemble the dashboard route in src/app/(dashboard)/ (<= 200 lines) with responsive breakpoints (1fr mobile, repeat(2, 1fr) at 600px, repeat(12, 1fr) at 900px+).
4. Run bun typecheck and bun lint, resolving all compiler, accessibility, and lint errors before completion.

Strategic Conclusions and Implementation Roadmap

Synthesizing visualization theory, mathematical layout systems, and frontend state architecture demonstrates that code generation quality in agentic workflows is governed by **upstream constraint specificity**[3]. When developers treat an AI coding agent purely as an algorithmic code generator at Level 4 of Munzner's Nested Model, the agent defaults to visually noisy, poorly structured interfaces that fail human perceptual tasks and accumulate severe state-management and accessibility debt[1]. Conversely, encoding Munzner's *What-Why-How* taxonomy, the $0.5R \leq G \leq 1.0R$ Bento Grid mathematics, WCAG 2.1 AA container semantics, and the React Query and Zustand separation rule into OpenCode's dual configuration surfaces (AGENTS.md and opencode.jsonc) and lazy-loaded SKILL.md modules transforms the agent into a deterministic systems architect[3].

To operationalize this workflow immediately, engineering teams should initialize their context surfaces by committing AGENTS.md and opencode.jsonc at the repository root while storing the visual ruleset inside .opencode/skill/bento-viz-dashboard/SKILL.md so OpenCode V2 loads core guardrails automatically while lazy-loading UI specifications only when needed[12]. During feature development, teams should enforce plan-first iteration by beginning every dashboard task in OpenCode's read-only plan mode (Tab), reviewing the agent's *What-Why-How* mapping table and Bento footprint matrix—and dragging reference wireframes directly into the terminal—before authorizing file mutations in build mode[8]. Throughout implementation, code reviewers should verify that all backend snake_case fields remain isolated inside src/lib/<domain>/transformers.ts, that Zustand stores remain under 50 lines with zero API state, and that no component file exceeds the 200-line ceiling[3]. Finally, pairing bun typecheck and axe-core accessibility checks with Alibaba's Open Code Review (ocr review) across local workspaces and GitHub Actions (/opencode) runners guarantees sustained architectural and perceptual quality across every pull request[13].

Works cited

1. Visualization Analysis Framework | PDF - Scribd,

https://www.scribd.com/document/617472103/vadallslides-2021
2. Card UI Design Prompts & Examples - Superdesign, https://superdesign.dev/library/category/card
3. AI Context Prompt Frontend - Hybrid Interactive Engineering, https://engineering.hybridinteractive.in/FRONTEND_LLM_PROMPT/
4. Bento Grid Dashboard Design: Complete Guide 2026 - Orbix Studio, https://www.orbix.studio/blogs/bento-grid-dashboard-design-aesthetics
5. The Four-Level Nested Model Revisited: Blocks and Guidelines, https://miriah.github.io/publications/nbgm-beliv.pdf
6. OpenCode AGENTS.md + opencode.jsonc - Dev Genius, https://blog.devgenius.io/opencode-agents-md-opencode-jsonc-architecture-precedence-and-workflow-9cb668e501e9
7. Bento grid layout explained with grid math and 12 real examples, https://www.setproduct.com/blog/bento-grid-layout-design-guide
8. Visualisation Analysis & Design by Tamara Munzner or ... - Data Rocks, https://www.datarocks.co.nz/blog/data-viz-bookshelf_visualization-analysis-design-tamara-munzner
9. A Nested Model for Visualization Design and Validation, https://www.semanticscholar.org/paper/A-Nested-Model-for-Visualization-Design-and-Munzner/e8cfef98279aabd43f65d465b304345aec15d9bb
10. Bento Design Skill File | TypeUI, https://www.typeui.sh/design-skills/bento
11. GitHub - anomalyco/opencode: The open source coding agent., https://github.com/anomalyco/opencode
12. opencode tweaking: mastering SKILL.md vs AGENTS.md - Tech blog, https://grigio.org/o-organization-mastering-skill-md-vs-agents-md/
13. Instructions - OpenCode, https://opencode.ai/v2/docs/instructions/
14. Visualization Analysis & Design What's Vis, and Why Do It? (Ch 1, https://www.cs.ubc.ca/~tmm/talks/vad/fullday-vis21-4x4.pdf
15. Tamara Munzner | alphaXiv, https://www.alphaxiv.org/@tamara-munzner
16. Nested Model (Ch 4) II, Visualization Analysis & Design, 2021, https://www.youtube.com/watch?v=Yq2IkdgWr9Q
17. Dashboard design principles that actually work | Setproduct Blog, https://www.setproduct.com/blog/effective-dashboard-design-principles
18. 230+ Shadcn Dashboard Blocks – Shadcnuikit.com, https://shadcnuikit.com/blocks/dashboard-ui
19. Monthly Ticket Volume Analytics Dashboard - Posterlet, https://posterlet.com/poster/monthly-ticket-volume-analytics-dashboard
20. Modern Bento Grid Layout CSS - CodeShack, https://codeshack.io/modern-bento-grid-layout-css/
21. Shadcn Separator - Base UI and Radix UI, https://shadcnstudio.com/docs/components/separator
22. KPI Card — Data Display - HaloUI, https://haloui.vercel.app/components/kpi-card
23. Best Shadcn UI Templates, Blocks & Themes (2026) - DesignRevision, https://designrevision.com/blog/best-shadcn-templates
24. Create an Exciting Dashboard Using Generative UI - Medium,

https://medium.com/@soumyajit.swain/create-an-exciting-dashboard-using-generative-ui-46b7aa054c05
25. Shadcn Charts Components: Examples, Use Cases & Best Practices, https://builtbyindies.com/stories/shadcn-charts-components-examples-use-cases-best-practices
26. awesome-gpt-image-prompt-2.5/docs/exclusive-web-1.md ... - GitHub, https://github.com/stretchcloud/awesome-gpt-image-prompt-2.5/blob/main/docs/exclusive-web-1.md
27. Accessible Data Visualization in R: Color Vision and Contrast, https://r-statistics.co/Accessible-Data-Visualization-in-R.html
28. Prompt Engineering | Mendix Documentation, https://docs.mendix.com/agents/prompt-engineering/
29. Download - OpenCode, https://opencode.ai/download
30. OpenCode - A powerful AI coding agent. Built for the terminal. - GitHub, https://github.com/opencode-ai/opencode
31. Intro | OpenCode, https://opencode.ai/v2/docs/
32. GitHub - OpenCode, https://opencode.ai/docs/github/
33. Intro | AI coding agent built for the terminal - OpenCode, https://opencode.ai/docs/
34. OpenCode: Free AI Coding Agent Setup Guide 2026 - YouTube, https://www.youtube.com/watch?v=20vcJmqAr4M
35. opencode/AGENTS.md at dev - GitHub, https://github.com/anomalyco/opencode/blob/dev/AGENTS.md
36. AGENTS.md, https://agents.md/
37. Rules | OpenCode, https://opencode.ai/docs/rules/
38. GitHub - alibaba/open-code-review: Secure, fast, efficient, battle, https://github.com/alibaba/open-code-review